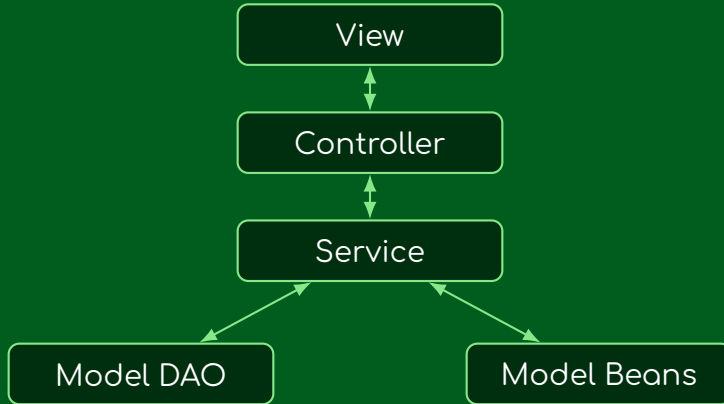


Green Refactoring of the 2chance Web Application: An Empirical Analysis of Energy Patterns and Performance

Sustainable Software Engineering
Green-in-Software Project

2Chance Web Application



View

HTML, CSS

INTERFACCIA UTENTE (UI)

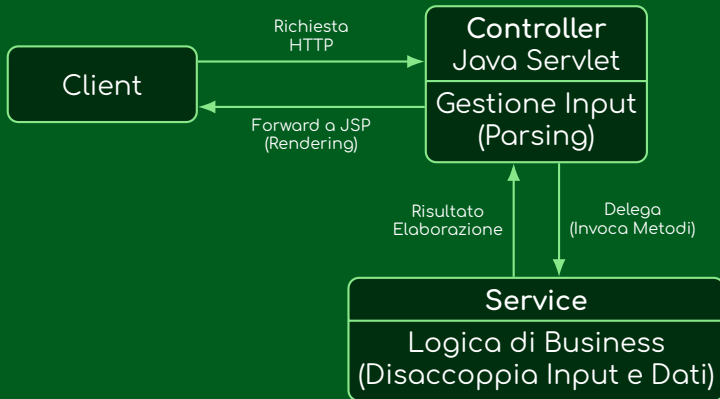
JavaScript

INTERATTIVITÀ LATO CLIENT E
ACQUISIZIONE DEGLI INPUT DELL'UTENTE

JavaServer Pages (JSP)

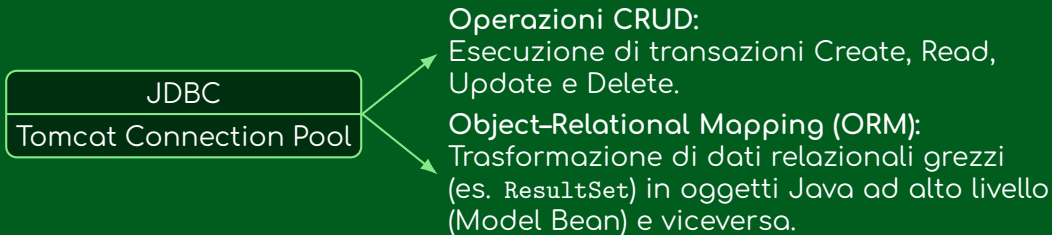
VISUALIZZARE DINAMICAMENTE DATI
FORNITI DAL CONTROLLER

Controller + Service



Model Data Access Object (DAO)

“Astrae e incapsula tutto l’accesso al database che contiene i dati persistenti.”



Software Dependability Improvements

Unit & Integration Testing

JUnit 5 → Automated Tests
Mockito → Mock Dipendenze Esterne
JaCoCo → Code Coverage
Pitest → Mutation Testing

Metodi Formali

JML → Specifica Formale
OpenJML → Verifica Formale

Deployment

Docker → Containerizzazione App
Docker Compose → Ambiente Multi-Container

GitHub Actions

GitGuardian → Secret Detection
Snyk → Software Composition Analysis
SonarQube Cloud → Static Analysis

Backend Energy Sustainability

Backend Energy Consumption

*Componente che compie il maggior sforzo computazionale →
Componente che consuma più energia.*

Componenti a maggior consumo:

- Gestione delle Interfacce (Controller)
- Business Logic (Service)
- Interrogazioni del Database (Model DAO | DBMS)

Backend Green Refactoring Plan

- Individuare tramite analisi statica e revisione del codice le porzioni di codice meno efficienti.
- Procedere al refactoring per ridurre lo spreco di risorse.
- Misurare il consumo energetico effettivo durante l'uso prima e dopo il refactoring.

Rule-Based Eco-Design Static Analysis



Docker

Poiché il progetto è containerizzato, l'esecuzione locale di SonarQube (in un container dedicato) in parallelo all'app risulta estremamente efficiente.

GitHub Actions

Poiché Creedengo opera come plugin per SonarQube, si integra direttamente nella pipeline di Static Application Security Testing (SAST) esistente.

Configurazione con Script Automatico di SonarQube con il Plugin Creedengo

1. Validazione dell'Ambiente

Verifica dell'esecuzione attiva del demone Docker sul sistema.



2. Scaricamento del Plugin Creedengo

Controllo della presenza locale del plugin ed eventuale download dinamico da GitHub.



3. Provisioning di SonarQube

Avvio di SonarQube in un container Docker configurando la corretta esposizione delle porte.



4. Iniezione del Plugin Creedengo

Copia del plugin all'interno del container, nella cartella riservata alle estensioni.



5. Riavvio ed Inizializzazione

Riavvio del container per forzare il caricamento del plugin e registrare le regole di eco-design.

Selezione delle Regole di Eco-Design

Massimizzare il Ritorno Ecologico
delle Attività di Refactoring

Ridurre il Rumore nelle
Metriche di Telemetria Statica

11 Regole Fondamentali del Ruleset Creedengo Java Selezionate

Regole Scelte per **Pertinenza Tecnologica** con l'Architettura di 2Chance Web App
*(App monolitica basata su Java Servlets e interazioni DB
tramite JDBC/DAO nativi, senza framework pesanti come Spring)*

Scansione iniziale (*Baseline*): rilevate complessivamente **64 violazioni di eco-design**
distribuite unicamente su 4 delle 11 regole (anti-pattern strutturali).

Model DAO Bottleneck Consumo Energetico

Collo di Bottiglia Energetico: i Model DAO

I Model DAO registrano i maggiori consumi energetici poiché si occupano della complessa comunicazione inter-processo (IPC) tra l'applicazione e il DBMS.



Soluzione: Refactoring Profondo

Per sanare le violazioni di Creedengo, risolvere problemi architetturali e massimizzare l'efficienza ecologica, si è optato per un **refactoring profondo** del sistema, concentrandosi specialmente sui **Model DAO** e sui **Model Beans** in cui sono state riscontrate la maggior parte delle problematiche di ecodesign.

Avoid getting the size of the collection in the loop (GCI3)

Problema

Invocare ripetutamente `.size()` nella guardia di un ciclo costringe la JVM a ricalcolare i limiti della collezione ad ogni iterazione, spreco di cicli di clock.



Soluzione

Estrarre l'invocazione in una variabile primitiva locale prima del ciclo agisce come *caching*. Ciò previene continue letture nell'Heap e permette alla CPU di usare i registri fisici, abilitando le ottimizzazioni JIT e annullando gli sprechi computazionali.

Avoid SQL request in loop (GCI72)

Problema

Query SQL in cicli iterativi generano enorme overhead DBMS, bloccando i thread Tomcat e impedendo il basso consumo della CPU. Inefficienze rilevate:

- **Assenza Batch Processing:** singole INSERT/UPDATE in ciclo forzano transazioni e commit continui per ogni record, frammentando l'I/O su disco e saturando la rete.
- **Problema N+1:** recuperare N ordini e i loro prodotti costa $1+N$ query distinte (1 per ordini, N per prodotti), moltiplicando roundtrip e attese I/O.



Soluzione

- **Risoluzione Scritture:** introdotte operazioni batch native JDBC (`addBatch()`, `executeBatch()`). I comandi accodati vengono inviati in un singolo pacchetto di rete ed eseguiti in un'unica transazione atomica, abbattendo lock e costi di connessione.
- **Risoluzione N+1:** le $1+N$ query sono state ridotte a 1 query aggregata (`LEFT JOIN`) in un solo roundtrip. L'assemblaggio senza duplicati è delegato alla RAM tramite `LinkedHashMap`, annullando del tutto gli sprechi di rete.

Do not call a function when declaring a for-type loop (GCI69)

Problema

Invocare funzioni **invarianti** (il cui risultato non cambia mai durante l'esecuzione) all'interno dei parametri di dichiarazione o di guardia di un ciclo forza la CPU a ricalcolare l'intero metodo ad ogni singola iterazione, causando un enorme e inutile dispendio di calcolo.



Soluzione

I risultati di queste funzioni sono stati estratti e salvati in **variabili primitive locali** *prima* dell'inizio del ciclo.

Ciò elimina alla radice migliaia di invocazioni ridondanti, fungendo da vero e proprio *caching* preventivo e consentendo alla JVM di ottimizzare pienamente l'esecuzione del blocco iterativo.

Don't use the query `SELECT * FROM` (GCI74)

Problema

L'uso sistematico di `SELECT *` costringe il database a pesanti operazioni di I/O su disco per estrarre **tutti i campi**, inclusi quelli inutili o enormi (es. immagini in *Base64*). Ciò genera traffico di rete sprecato e satura la JVM con massicce allocazioni di oggetti temporanei.



Soluzione

Le query sono state riscritte esplicitando nella proiezione SQL **unicamente le colonne necessarie**.

Il taglio del *payload* di rete alleggerisce la fase di mappatura ad oggetti (ORM), abbattendo le allocazioni nella memoria Heap e minimizzando l'intenso stress energetico causato dai cicli continui di **Garbage Collection**. L'allineamento dei risultati ai nomi simbolici delle colonne aumenta inoltre la resilienza strutturale del codice.

Unit & Integration Testing Incrementali

Unit Testing Incrementale

Ogni classe è stata rifattorizzata singolarmente seguendo un approccio incrementale. Al termine delle modifiche, la relativa suite di **unit testing** è stata eseguita tramite Maven e JUnit 5 per isolare e correggere immediatamente eventuali errori semantici o disallineamenti nei mock (es. per l'introduzione di query batch o letture per nome colonna).



Integration Testing Incrementale

Una volta validato il singolo componente, è stata lanciata l'intera suite globale di **integration testing** (composta da oltre 500 test).

Questo rigoroso processo di verifica a doppio passaggio, ripetuto per ogni classe, ha scongiurato qualsiasi regressione e certificato l'assoluta integrità della logica di business prima di validare le ottimizzazioni energetiche.

Verifica Risoluzione Anti-Pattern Energetici Post-Refactoring con Creedengo

Tutte le **64** eco-vulnerabilità sono state risolte ed i relativi fault sono stati chiusi con successo, venendo contrassegnati da Creedengo con lo stato **CLOSED/FIXED**.



La completa risoluzione di questi fault garantisce che l'applicazione non presenti inefficienze strutturali latenti a livello di CPU-bound e I/O-bound, allineandosi con i rigorosi **requisiti di sostenibilità energetica** previsti per il sistema 2Chance.

Verifica Ottimizzazioni Energetiche e Performance

branch `baseline` VS branch `sse-improvements`



Microbenchmarking (JMH)

Profilazione Consumo Energetico (EnergiBridge)



System Performance Testing (JMeter)

Profilazione Consumo Energetico (EnergiBridge)

EnergiBridge - Profilazione Consumo Energetico

EnergiBridge

- **Scopo:** raccolta empirica delle metriche energetiche, campionando il consumo a intervalli regolari durante l'esecuzione di un processo.
- **Interfaccia:** riga di comando (CLI), snella e programmabile.
- **Output:** esportazione dei dati aggregati in formato CSV.



Interazione Hardware su Windows

Il tool monitora l'impatto energetico interfacciandosi direttamente con i sensori fisici dell'architettura. Per farlo su Windows, integra il plugin **LibreHardwareMonitor**, un componente essenziale che permette al software di leggere a basso livello i dati estratti dai registri della CPU.

Questa compatibilità risulta fondamentale per il progetto 2Chance, le cui rigorose misurazioni verranno effettuate su sistema operativo Windows con processore **AMD Ryzen AI 9 HX 370**.

Microbenchmarking Energetico sui DAO

JMH + EnergiBridge

Selezione dei Target (DAO)

Dall'analisi di *Creedengo* è emerso che gli anti-pattern responsabili dell'elevato consumo energetico erano concentrati in tre classi: `ProdottoDAO`, `RecensioneDAO` e `OrdineDAO`. Queste sono state scelte come target per il microbenchmarking.



Sincronizzazione Automatizzata in Python

Per misurare simultaneamente le performance temporali (JMH) e l'assorbimento energetico (EnergiBridge) di ogni metodo di queste classi, è stato sviluppato lo script Python `run_jmh_energy_profiler.py`.

Questo script funge da "regista", orchestrando l'intero processo di isolamento e profilazione hardware attraverso **4 fasi analitiche sequenziali**.

Fase 1: Isolamento Totale con Nuova JVM

Il Problema delle Interferenze

JMH normalmente esegue test multipli all'interno della stessa *Java Virtual Machine* (JVM). Questo rischia di alterare severamente i consumi energetici, perché un test potrebbe subire il carico del *Garbage Collector* innescato dal test precedente, sporcando i dati.



Avvio Isolato

Lo script impone l'avvio di una JVM **completamente nuova e pulita** per ogni singolo metodo testato, incapsulandola all'interno del processo di EnergiBridge.

Ciò previene l'inquinamento della memoria *Heap* e l'accavallamento dei thread, garantendo che i Joule registrati appartengano **esclusivamente** al frammento di codice in esame.

Fase 2: Configurazione Microbenchmarking

L'Ottimizzazione JIT del Bytecode

Il codice Java è pre-compilato in un formato intermedio (*bytecode*). Durante l'avvio, il compilatore JIT (*Just-In-Time*) della JVM lo traduce al volo in linguaggio macchina, applicando pesanti ottimizzazioni per l'hardware. Questo intenso processo di analisi e ottimizzazione del codice assorbe enormi quantità di energia che falsificherebbero i risultati.



Warmup e Misurazione a Singolo Thread

Lo script imposta JMH per eseguire prima **5 iterazioni a vuoto** (*warmup*). Queste iterazioni servono esclusivamente a concedere alla JVM il tempo e l'energia per profilare e ottimizzare a fondo il *bytecode* in background.

Solo quando il codice è stato completamente ottimizzato dal JIT, lo script lancia le **5 iterazioni di misurazione reale** a singolo thread, registrando il vero costo energetico dell'algoritmo a regime.

Fase 3: Pulizia Dati e Filtro RAPL

Rumore dei Sensori Hardware

I sensori fisici integrati nel processore (registri *RAPL*) possono generare rumore, cioè valori anomali matematicamente impossibili, come misurazioni con energia negativa, o picchi anomali di assorbimento in Watt che vanno oltre le capacità fisiche della CPU e dei singoli Core.



Filtraggio Matematico (TDP)

Lo script applica un rigoroso filtro, scartando le letture in cui l'energia registrata (ΔE) divisa per il tempo (Δt) supera il TDP (*Thermal Design Power*) massimo fisicamente erogabile dalla CPU AMD usata per i test:

$$0 \leq \Delta E_{\text{CPU}} \leq 55,0 \times \Delta t \quad \text{e} \quad 0 \leq \Delta E_{\text{Core}} \leq 45,0 \times \Delta t$$

Ciò garantisce la totale **affidabilità empirica** dei misuramenti.

Fase 4: Merging e Report Finale

Disallineamento delle Metriche

Le metriche estratte dai due strumenti possiedono nature differenti: JMH produce log misurando il tempo di esecuzione in **millisecondi per operazione**, mentre EnergiBridge genera file isolati misurando l'assorbimento in **Joule e Watt**.



Riconciliazione ed Export CSV

L'ultima mansione dello script Python è quella di scansionare ed unire temporalmente e matematicamente i log isolati dei due software.

Viene generato un **singolo report CSV finale** che quantifica con precisione assoluta il consumo netto per operazione (**Joule/op**), affiancato dalla potenza media erogata (**W**), dai picchi di RAM allocata e dal carico della CPU.